

Determinación de la Actitud mediante Giróscopos



ARA SIID
División Electrónica

Nicolás Eduardo Real
Incicco, Sebastian Javier
Sala De Ferraris, Facundo Ignacio

2014

Índice

Objetivo	3
Introducción	4
Descripción general de las ternas de referencia involucradas	4
Ecuación de actitud	5
Métodos de resolución	6
Métodos de resolución según la forma de la ecuación del cono	6
Implementación en Matlab	11
Resultados de las simulaciones	15
Conclusiones	19
Apéndice A: Álgebra de Cuaterniones	20
Operaciones	20
Suma	20
Multiplicación	20
Opuesto	20
Conjugado	21
Relación con eje y ángulo de Euler	21
Rotaciones elementales de Euler	21
Bibliografía / Referencias	23

Objetivo

El presente trabajo es una aplicación directa del informe sobre "Métodos de Resolución Inercial 3D", y es también una simplificación. Utilizando una unidad inercial de mediciones constituida por tres giróscopos, se buscará determinar la orientación de un vehículo que navega libre.

Introducción

En esta sección se explicará brevemente cómo es el sistema de navegación, las ternas en juego y las magnitudes a determinar. Podrá apreciarse la similitud de esta sección con la correspondiente en el informe técnico sobre "Métodos de Resolución Inercial 3D".

Descripción general de las ternas de referencia involucradas

Considerando las siguientes ternas de referencia. Mayor detalle ver referencia [1].

- ✓ **e**: Terna terrestre, solidaria a la Tierra, con su origen en el centro del elipsoide de referencia WGS-84. El eje X pasa a través del meridiano cero y el eje Z coincide con el eje de rotación de la Tierra apuntando hacia el Polo Norte.
- ✓ **i**: Terna inercial alineada con la **e** al comienzo de la navegación.

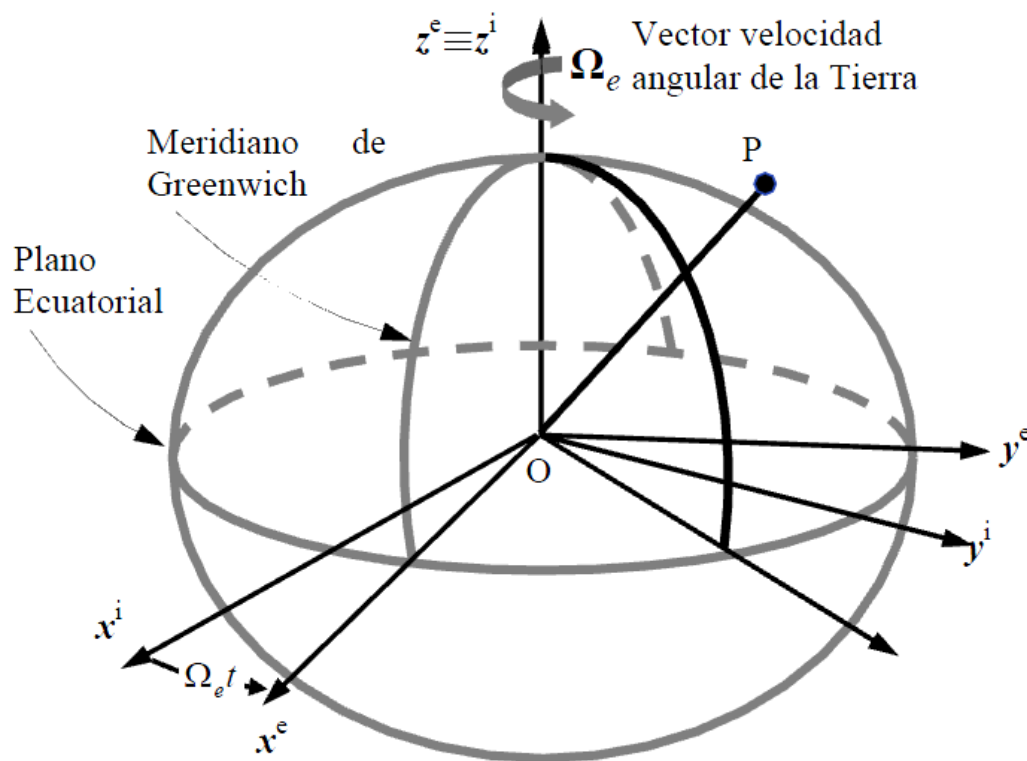


Figura 1. Elipsoide normal con las ternas inercial y terrestre.

- ✓ **g**: Terna geográfica local (ENU) con el eje X hacia el Este, Y hacia el Norte y U perpendicular (saliente) al plano tangente local del elipsoide WGS-84.



b: Terna fija al vehículo.

Ecuación de actitud

$$\dot{q}_b^e = \frac{1}{2} q_b^e \cdot \begin{bmatrix} \mathbf{w}_{ib}^b \\ 0 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} \mathbf{\Omega}^e \\ 0 \end{bmatrix} \cdot q_b^e$$

Métodos de resolución

Existen varias alternativas para obtener los parámetros de actitud buscados. En particular utilizaremos en este trabajo métodos de integración que aprovechen la forma particular de la ecuación del cono.

Métodos de resolución según la forma de la ecuación del cono

Se definen nuevamente al igual que en el informe sobre "Métodos de Resolución Inercial 3D", tres intervalos de tiempo encastrados. Un intervalo corto **TI**, que define la tasa de entrada de datos de los instrumentos inerciales; un intervalo más extenso **Ts**, que determina la tasa de salida de los parámetros estimados de navegación; y un intervalo intermedio **Tm**, cuyo fin quedará explicado al analizar el algoritmo. Los tres intervalos se relacionan de la siguiente manera:

$$Tm = L \cdot TI; \quad Ts = M \cdot Tm$$

A su vez, se utilizan tres subíndices **k, j, i** relacionados con **Ts, Tm** y **TI** respectivamente, para referirse a un punto cualquiera en el tiempo.

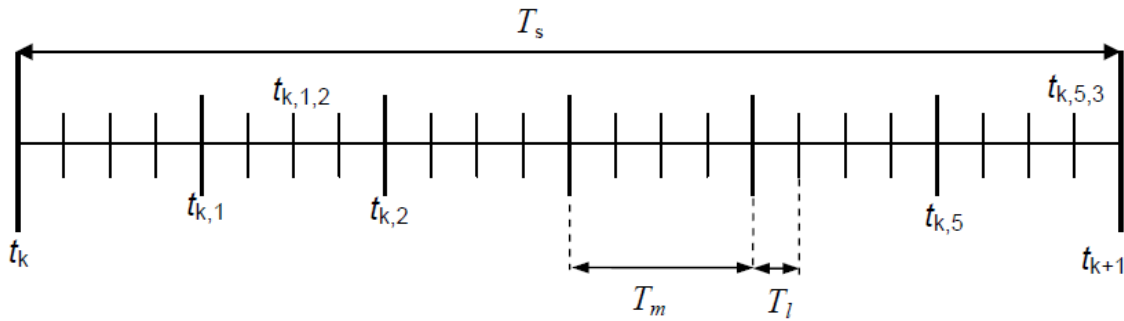


Figura 3. Períodos de tiempo encastrados, que se utilizan en el algoritmo ad-hoc.

Esto se debe a que las cantidades involucradas en las ecuaciones de navegación cambian con dinámicas muy variadas. Por ejemplo, podemos descomponer la actitud en los siguientes factores:

$$q_b^e(k+1) = q_{e(k)}^{e(k+1)} \cdot q_b^e(k) \cdot q_{b(k+1)}^{b(k)}$$

La componente de la actitud correspondiente al movimiento de la terna **e** entre dos instantes **k** consecutivos tiene una dinámica lenta. En cambio, la componentes de actitud correspondiente a la terna **b** en dos instantes **k** consecutivos puede llegar a variar notablemente. Estas diferencias en las dinámicas de las componentes de actitud son las que dan origen a la estrategia de los tres periodos encastrados.

Se define el ángulo de rotación $\phi_{k,j,i}^b$ que representa las rotaciones de la terna **b** en el instante $t_{k,j}$ al instante $t_{k,j,i}$. Para ello se integra la ecuación del cono, en la cual se introduce una aproximación truncando términos de segundo orden, por tratarse de un ángulo de rotación muy pequeño.

$$\begin{aligned}\dot{\phi}(t) &= \omega_{ib}^b + \frac{1}{2} \cdot \phi \times \omega_{ib}^b + \frac{1}{\|\phi\|^2} \cdot \left[1 - \frac{\|\phi\| \cdot \sin(\|\phi\|)}{2 \cdot (1 - \cos(\|\phi\|))} \right] \cdot \phi \times \omega(\phi \times \omega_{ib}^b) \\ \Rightarrow \begin{cases} \dot{\phi}(t) \approx \omega_{ib}^b + \frac{1}{2} \cdot \alpha(t) \times \omega_{ib}^b \\ \dot{\alpha}(t) = \omega_{ib}^b \end{cases} \\ \phi_{k,j,i}^b &= \int_{t_{k,j}}^{t_{k,j,i}} \left(\omega_{ib}^b + \frac{1}{2} \cdot \alpha(\tau) \times \omega_{ib}^b \right) d\tau = \underbrace{\int_{t_{k,j}}^{t_{k,j,i}} \omega_{ib}^b(\tau) d\tau}_{\alpha_{k,j,i}} + \underbrace{\int_{t_{k,j}}^{t_{k,j,i}} \frac{1}{2} \cdot \alpha(\tau) \times \omega_{ib}^b(\tau) d\tau}_{\beta_{k,j,i}} \\ \phi_{k,j,i+1}^b &= \underbrace{\phi_{k,j,i}^b}_{\alpha_{k,j,i} + \beta_{k,j,i}} + \underbrace{\int_{t_{k,j,i}}^{t_{k,j,i+1}} \omega_{ib}^b(\tau) d\tau}_{\Delta\alpha_{k,j,i+1}} + \underbrace{\frac{1}{2} \cdot \int_{t_{k,j,i}}^{t_{k,j,i+1}} \alpha(\tau) \times \omega_{ib}^b(\tau) d\tau}_{\Delta\beta_{k,j,i+1}} \\ \alpha_{k,j,i+1} &= \alpha_{k,j,i} + \Delta\alpha_{k,j,i+1} \\ \beta_{k,j,i+1} &= \beta_{k,j,i} + \Delta\beta_{k,j,i+1}\end{aligned}$$

Obsérvese que la cantidad $\Delta\alpha_{k,j,i+1}$ es información de entrada al algoritmo y es la medición entregada por los giróscopos, integrada en un intervalo de tiempo **TI**. Por lo tanto, sólo queda por determinar:

$$\Delta\beta_{k,j,i+1} = \frac{1}{2} \cdot \int_{t_{k,j,i}}^{t_{k,j,i+1}} \alpha(\tau) \times \omega_{ib}^b(\tau) d\tau = \frac{1}{2} \cdot \alpha_{k,j,i} \times \Delta\alpha_{k,j,i+1} + \underbrace{\frac{1}{2} \cdot \int_{t_{k,j,i}}^{t_{k,j,i+1}} \Delta\alpha_{k,j,i}(\tau) \times \omega_{ib}^b(\tau) d\tau}_I$$

Nuevamente aplicando la hipótesis de que $\omega_{ib}^b(\tau)$ varía linealmente a lo largo de dos intervalos **TI** consecutivos:

$$\begin{aligned}\omega_{ib}^b(\tau) &\approx a_i + b_i \cdot (\tau - t_{k,j,i}), \quad t_{k,j,i-1} \leq \tau \leq t_{k,j,i+1} \\ \left. \begin{aligned} \Delta\alpha_{k,j,i} &= a_i \cdot Tl - \frac{1}{2} b_i \cdot Tl^2 \\ \Delta\alpha_{k,j,i+1} &= a_i \cdot Tl + \frac{1}{2} b_i \cdot Tl^2 \end{aligned} \right\} \rightarrow \begin{aligned} a_i &= \frac{\Delta\alpha_{k,j,i+1} + \Delta\alpha_{k,j,i}}{2 \cdot Tl} \\ b_i &= \frac{\Delta\alpha_{k,j,i+1} - \Delta\alpha_{k,j,i}}{Tl^2} \end{aligned}$$

$$\begin{aligned}
I &= \frac{1}{2} \cdot \int_{t_{k,j,i}}^{t_{k,j,i+1}} \Delta \alpha_{k,j,i}(\tau) \times \omega_{ib}^b(\tau) d\tau = \frac{1}{2} \cdot \int_{t_{k,j,i}}^{t_{k,j,i+1}} \left[a_i \cdot (\tau - t_{k,j,i}) + \frac{1}{2} b_i \cdot (\tau - t_{k,j,i})^2 \right] \times [a_i + b_i \cdot (\tau - t_{k,j,i})] d\tau = \\
&= \frac{1}{2} \cdot \int_{t_{k,j,i}}^{t_{k,j,i+1}} \left[a_i \times b_i (\tau - t_{k,j,i})^2 + \frac{1}{2} b_i \times a_i \cdot (\tau - t_{k,j,i})^2 \right] d\tau = \frac{1}{4} \cdot a_i \times b_i \frac{(\tau - t_{k,j,i})^3}{3} \Big|_{t_{k,j,i}}^{t_{k,j,i+1}} = \\
&= \frac{1}{12} \cdot a_i \times b_i \cdot Tl^3 = \frac{1}{12} \cdot \left(\frac{\Delta \alpha_{k,j,i+1} + \Delta \alpha_{k,j,i}}{2 \cdot Tl} \right) \times \left(\frac{\Delta \alpha_{k,j,i+1} - \Delta \alpha_{k,j,i}}{Tl^2} \right) \cdot Tl^3 = \\
&= \frac{1}{12} \cdot \Delta \alpha_{k,j,i} \times \Delta \alpha_{k,j,i+1}
\end{aligned}$$

Entonces pasando en limpio:

$$\begin{aligned}
\Delta \beta_{k,j,i+1} &= \frac{1}{2} \cdot \int_{t_{k,j,i}}^{t_{k,j,i+1}} \alpha(\tau) \times \omega_{ib}^b(\tau) d\tau = \frac{1}{2} \cdot \alpha_{k,j,i} \times \Delta \alpha_{k,j,i+1} + \frac{1}{12} \cdot \Delta \alpha_{k,j,i} \times \Delta \alpha_{k,j,i+1} = \\
&= \frac{1}{2} \cdot \left(\alpha_{k,j,i} + \frac{1}{6} \cdot \Delta \alpha_{k,j,i} \right) \times \Delta \alpha_{k,j,i+1}
\end{aligned}$$

Pseudocódigo del algoritmo

El pseudocódigo presentado a continuación si bien no agrega conocimiento ni información alguna, ordena los resultados que fueron mostrados anteriormente.

Cuaternión que puede calcularse a priori:

$$q^e = \begin{bmatrix} \frac{\Omega^e}{\Omega} \cdot \sin\left(\frac{\Omega}{2}\right) \\ \cos\left(\frac{\Omega}{2}\right) \end{bmatrix}$$

Condiciones iniciales (inicialización de las variables del algoritmo):

$$q_{b(0)}^{e(0)} \rightarrow q_{b(k,j+1)}^{e(k)} = q_{b(0)}^{e(0)} \quad \begin{matrix} \Delta\alpha_{k,j,i} = 0 \\ \Delta v_{k,j,i} = 0 \end{matrix}$$

Algoritmo:

$$\text{for } k = \frac{t_0}{Ts} \rightarrow \frac{t_f}{Ts}$$

$$q_{b(k,j)}^{e(k)} = [0 \ 0 \ 0 \ 1]^T$$

$$\text{for } j = 0 \rightarrow M-1$$

$$\alpha_{k,j,i} = 0, \quad \phi_{k,j,i} = 0$$

$$\text{for } i = 1 \rightarrow L$$

$$\left. \begin{matrix} \Delta\alpha_{k,j,i-1} = \Delta\alpha_{k,j,i} \\ \alpha_{k,j,i-1} = \alpha_{k,j,i} \\ \phi_{k,j,i-1} = \phi_{k,j,i} \end{matrix} \right\} \begin{matrix} \text{se conservan los valores del instante} \\ t_{k,j,i-1} \text{ para utilizarse en el instante } t_{k,j,i} \end{matrix}$$

$$\Delta\alpha_{k,j,i} \left. \begin{matrix} \text{se toma la medición} \\ \text{de los giróscopos} \end{matrix} \right\}$$

$$\alpha_{k,j,i} = \alpha_{k,j,i-1} + \Delta\alpha_{k,j,i}$$

$$\Delta\beta_{k,j,i} = \frac{1}{2} \cdot \left(\alpha_{k,j,i-1} + \frac{1}{6} \cdot \Delta\alpha_{k,j,i-1} \right) \times \Delta\alpha_{k,j,i}$$

$$\phi_{k,j,i}^b = \phi_{k,j,i-1}^b + \Delta\alpha_{k,j,i} + \Delta\beta_{k,j,i}$$

$$\phi_{k,j,i-1}^b \rightarrow q_{b(k,j,i-1)}^{b(k,j)}$$

end

$$\left. \begin{matrix} q_{b(k,j)}^{b(k)} = q_{b(k,j+1)}^{b(k)} \\ q_{b(k,j)}^{e(k)} = q_{b(k,j+1)}^{e(k)} \end{matrix} \right\} \begin{matrix} \text{se conservan los valores del instante} \\ t_{k,j} \text{ para utilizarse en el instante } t_{k,j+1} \end{matrix}$$

$$\phi_{k,j,i}^b \rightarrow q_{b(k,j,i)}^{b(k,j)} = \left[\begin{array}{c} \frac{\phi_{k,j,i}^b}{\|\phi_{k,j,i}^b\|} \cdot \sin\left(\frac{\|\phi_{k,j,i}^b\|}{2}\right) \\ \cos\left(\frac{\|\phi_{k,j,i}^b\|}{2}\right) \end{array} \right] = q_{b(k,j+1)}^{b(k,j)} \left. \vphantom{\frac{\phi_{k,j,i}^b}{\|\phi_{k,j,i}^b\|}} \right\} i = L \Rightarrow t_{k,j,i} = t_{k,j+1}$$

$$q_{b(k,j+1)}^{b(k)} = q_{b(k,j)}^{b(k)} \cdot q_{b(k,j+1)}^{b(k,j)}$$

$$q_{b(k,j+1)}^{e(k)} = q_{b(k,j)}^{e(k)} \cdot q_{b(k,j+1)}^{b(k,j)}$$

end

$$q_{b(k+1)}^{e(k)} = q_{b(k,j+1)}^{e(k)} \left. \vphantom{q_{b(k,j+1)}^{e(k)}} \right\} j = M - 1 \Rightarrow t_{k,j} = t_{k+1}$$

$$q_{b(k+1)}^{e(k+1)} = q_b^e(k+1) = q^e \cdot q_{b(k+1)}^{e(k)}$$

$$q_{b(k,j+1)}^{e(k)} = q_b^e(k+1) \left\{ \begin{array}{l} \text{guardo el valor del cuaternión para inicializar} \\ \text{como valor inicial del próximo ciclo} \end{array} \right.$$

end

Implementación en Matlab

En Matlab se implementaron dos algoritmos. El primero de ellos es un sintetizador de datos. Genera las mediciones que arrojarían una terna de giróscopos a bordo del vehículo, así como también la orientación teórica precisa. Esto permite utilizar dicha información para la verificación del algoritmo antes descrito, e implementado en un segundo script de Matlab. Para la síntesis de los datos se parte de una función arbitraria que genera mediciones absolutas de los tres giróscopos $\omega_{ib}^b(t)$ agrupados en forma de vector columna. Por otro lado, se implementa en una función diferente la educación de navegación, que en este caso, abarca solamente la ecuación del coneo en su versión matricial en terna ECEF:

$$\dot{q}_b^e(t) = \frac{1}{2} q_b^e \cdot \begin{bmatrix} \omega_{ib}^b(t) \\ 0 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} \Omega^e \\ 0 \end{bmatrix} \cdot q_b^e(t)$$

Se utiliza la función de Matlab *ode23*(.) que implementa un método de resolución numérico de la ecuación diferencial antes mencionada. Como resultado se obtiene el cuaternión de orientación q_b^e para diferentes instantes de tiempo conocidos. Esto constituye la solución de la ecuación del coneo y es también información sobre la orientación teórica desarrollada por el vehículo. Es decir, de esta información, y conociendo la posición de ubicación global del vehículo, pueden determinarse los ángulos de Euler (Yaw ψ , Pitch θ y Roll φ) teóricos de la siguiente forma:

$$q_b^g = q_e^g \cdot q_b^e$$

Luego a partir del cuaternión q_b^g puede construirse la matriz de rotación C_b^g , de la cual pueden extraerse los ángulos de Euler.

$$C_b^g(1,1) = \cos(\theta) \cdot \cos(\psi)$$

$$C_b^g(1,2) = \cos(\theta) \cdot \sin(\psi)$$

$$C_b^g(1,3) = -\sin(\theta)$$

$$C_b^g(2,1) = -\cos(\varphi) \cdot \sin(\psi) + \sin(\varphi) \cdot \sin(\theta) \cdot \cos(\psi)$$

$$C_b^g(2,2) = \cos(\varphi) \cdot \cos(\psi) + \sin(\varphi) \cdot \sin(\theta) \cdot \sin(\psi)$$

$$C_b^g(2,3) = \cos(\theta) \cdot \sin(\varphi)$$

$$C_b^g(3,1) = \sin(\varphi) \cdot \sin(\psi) + \cos(\varphi) \cdot \sin(\theta) \cdot \cos(\psi)$$

$$C_b^g(3,2) = -\sin(\varphi) \cdot \cos(\psi) + \cos(\varphi) \cdot \sin(\theta) \cdot \sin(\psi)$$

$$C_b^g(3,3) = \cos(\theta) \cdot \cos(\varphi)$$

$$\Rightarrow \begin{cases} \psi = \arctan\left(\frac{C_b^g(1,2)}{C_b^g(1,1)}\right) \\ \theta = \arcsin(-C_b^g(1,3)) \\ \varphi = \arctan\left(\frac{C_b^g(2,3)}{C_b^g(3,3)}\right) \end{cases}$$

Los $\arctan(\cdot)$ deben considerarse con el signo según los cuatro cuadrantes. En Matlab, eso se implementa fácilmente con la función $\text{atan2}(\cdot)$.

Sin embargo, hemos visto en las secciones anteriores que el algoritmo a implementar utiliza como datos de entrada diferenciales de arco $\Delta\alpha_{k,j,i}$. Dichos diferenciales se generan integrando numéricamente la función que genera los datos de velocidad angular mencionada en un principio $\omega_{ib}^b(t)$. La función para la integración numérica en Matlab es $\text{quadv}(\cdot)$.

Una vez dicho todo esto, se procede a ejecutar el algoritmo de generación de datos para un periodo de 100 segundos. A continuación se muestran gráficos de las componentes del cuaternión de orientación teórico q_b^e y los ángulos de Euler teóricos:

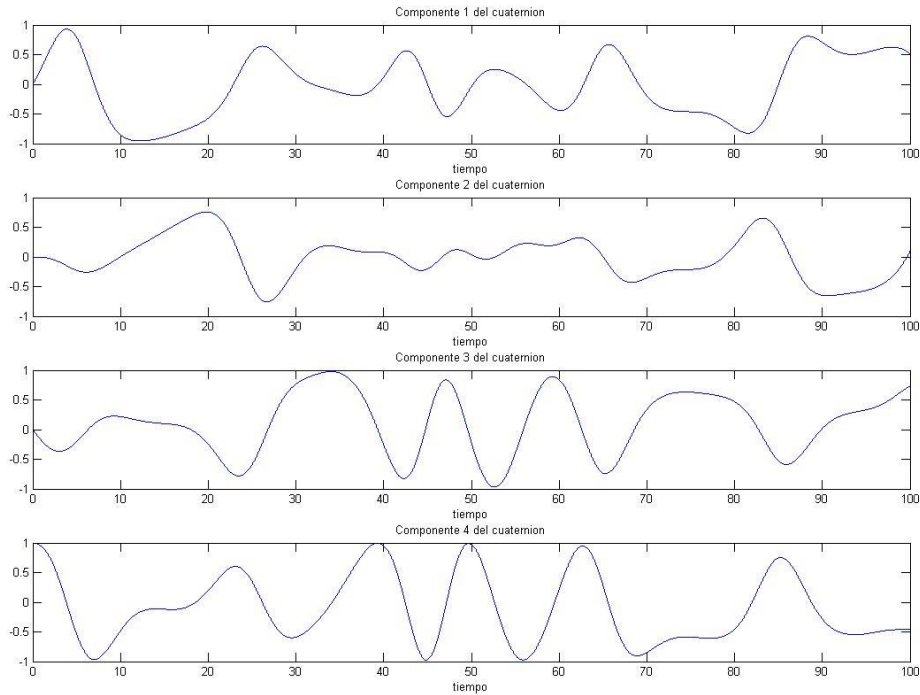


Figura 4 – Componentes del cuaternión teórico q_b^e

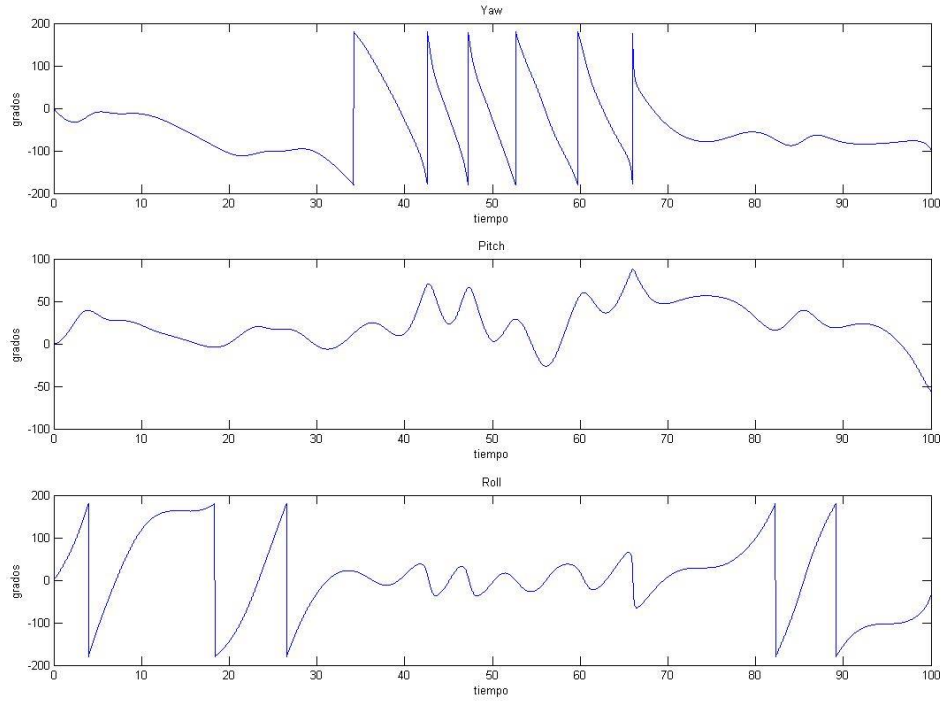


Figura 5 – Ángulos de Euler teóricos

Por otro lado, una vez ejecutado el script que genera los datos, los mismos se almacenan en memoria como variables. Dichas variables son utilizadas por el segundo script implementado en Matlab, encargado de la implementación del algoritmo de resolución mencionado en las secciones anteriores. Sin embargo, este segundo script no sólo implementa el algoritmo tal y como se lo describió anteriormente sino también una versión similar pero utilizando matrices en lugar de cuaterniones. El objetivo es comparar la performance de estas dos versiones y contrastar las estimaciones con los datos teóricos.

El algoritmo de multiplicación de matrices funciona de la siguiente manera. Se consideran las mediciones de los tres giróscopos por separado, y con cada una de ellas se construye una matriz de rotación alrededor del eje del giróscopo correspondiente:

$$\Delta\alpha = \begin{bmatrix} -\Delta\alpha_x \\ -\Delta\alpha_y \\ -\Delta\alpha_z \end{bmatrix}$$

Los signos negativos que anteceden a cada componente se deben simplemente a una convención.

$$\Delta\alpha_x \rightarrow C_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\Delta\alpha_x) & \sin(\Delta\alpha_x) \\ 0 & -\sin(\Delta\alpha_x) & \cos(\Delta\alpha_x) \end{bmatrix}$$

$$\Delta\alpha_y \rightarrow C_y = \begin{bmatrix} \cos(\Delta\alpha_y) & 0 & -\sin(\Delta\alpha_y) \\ 0 & 1 & 0 \\ \sin(\Delta\alpha_y) & 0 & \cos(\Delta\alpha_y) \end{bmatrix}$$

$$\Delta\alpha_z \rightarrow C_z = \begin{bmatrix} \cos(\Delta\alpha_z) & \sin(\Delta\alpha_z) & 0 \\ -\sin(\Delta\alpha_z) & \cos(\Delta\alpha_z) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Luego, basándonos en el hecho de que las pequeñas rotaciones son conmutativas, podemos construir una matriz de rotación total simplemente multiplicando las tres matrices:

$$C_{rot} = C_x \cdot C_y \cdot C_z$$

Una vez calculada dicha matriz, y teniendo en cuenta que el algoritmo es recursivo (es decir, utiliza la matriz de rotación calculada en el paso anterior), la operación que calcula la matriz actual de rotación es la siguiente:

$$C_b^e(k+1) = C_e \cdot C_b^e(k) \cdot C_{rot}$$

Procedemos a ejecutar el segundo script y analizaremos los resultados obtenidos.

Resultados de las simulaciones

En cada corrida se evalúan los resultados obtenidos por ambos métodos de resolución. No se considerarán para estos casos errores de medición en giróscopos de ningún tipo. Es decir, no existe ruido blanco de medición ni sesgo en las mediciones tomadas.

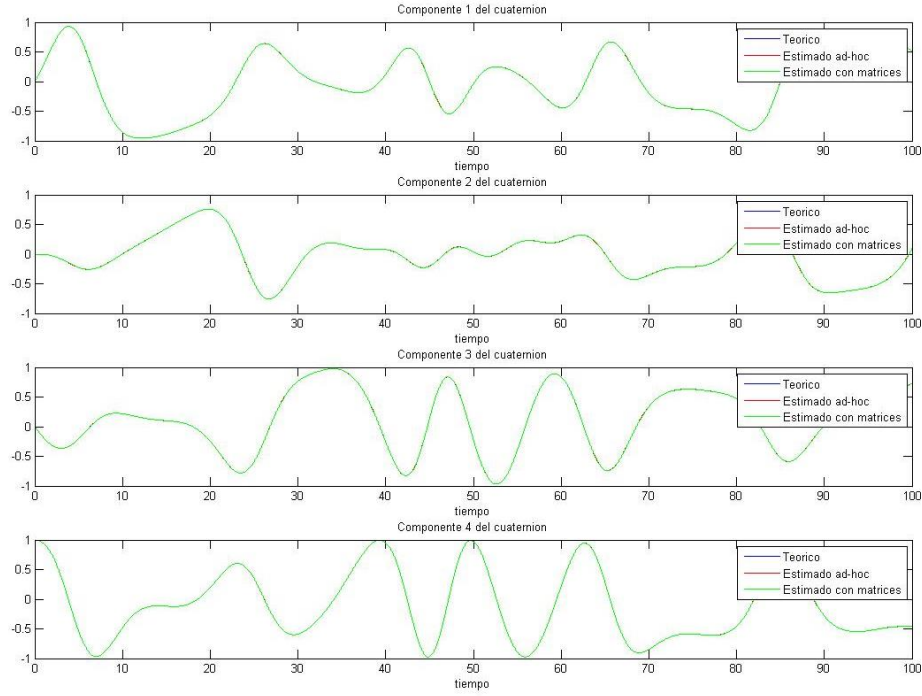


Figura 6 – Componentes del cuaternión q_b^e

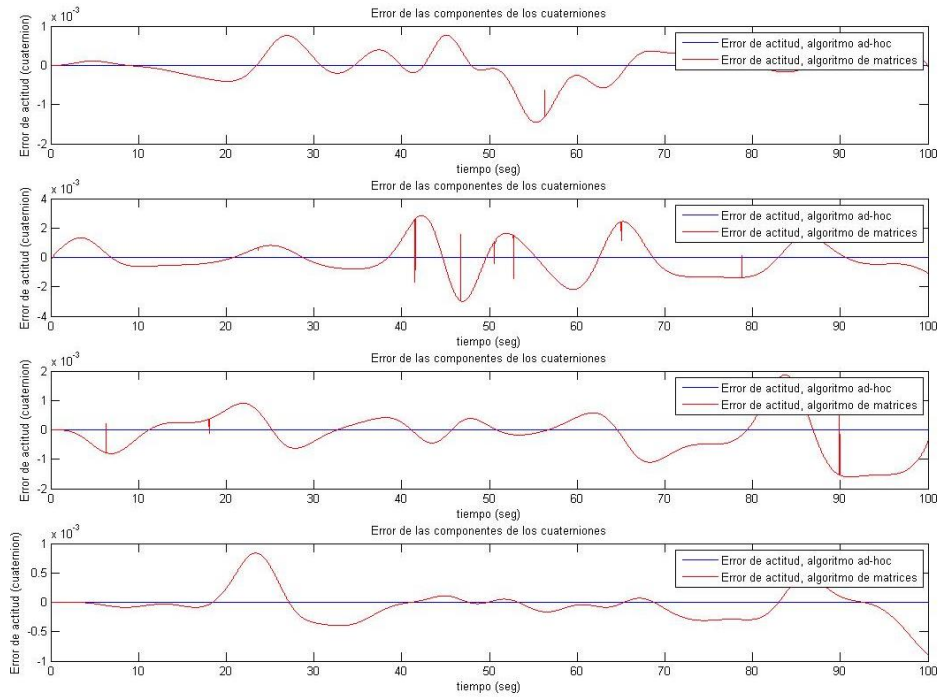


Figura 6 – Errores de las componentes del cuaternión q_b^e

En las figuras anteriores se observan las componentes del cuaternión q_b^e y los errores correspondientes. Se puede observar que el error de dichas componentes se mantiene bajo en ambos métodos de resolución. Sin embargo, el método de los cuaterniones parece ser mas preciso que el de multiplicación de matrices. Cabe aclarar que para poder realizar una comparación entre ambos métodos, en el método de las matrices fue necesario convertir cada matriz de rotación de cada instante de tiempo a su equivalente cuaternión de rotación.

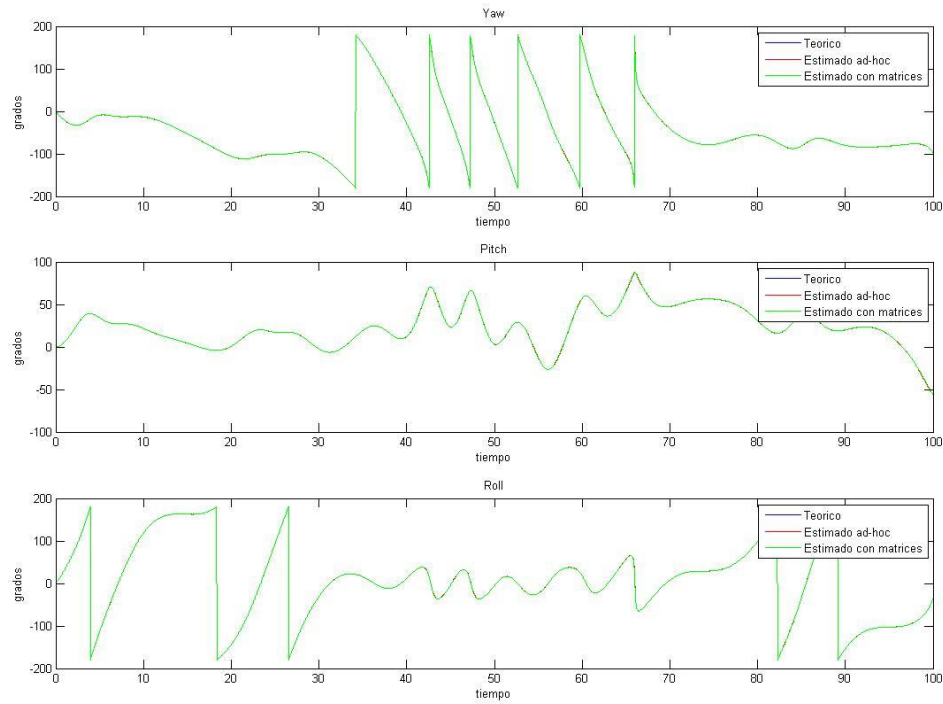


Figura 7 – Ángulos de Euler

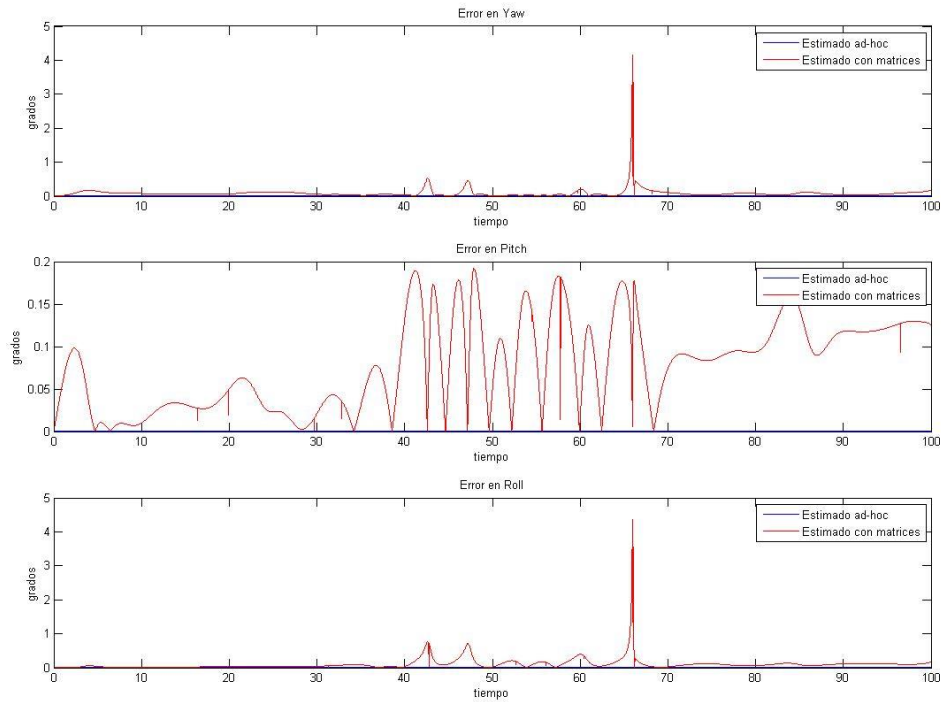


Figura 8 – Error en los ángulos de Euler

Luego podemos comparar los ángulos de Euler obtenidos por ambos métodos. Se observan errores del orden de los 0.2° excepto en determinados instantes de tiempo. Dichos errores se deben a la singularidad que se produce cuando el ángulo Pitch es 90°. En ese caso, la tangente de Pitch $\tan(\theta)$ tiende a infinito. Esto hace que cualquier error en el cálculo de las componentes de los cuaterniones o matrices, por más pequeño que sea, produzca un error mucho más significativo en el cálculo de los ángulos de Euler. Puede observarse en las figuras anteriores que los errores en los ángulos de Euler crecen cuando Pitch se acerca a 90°. En cambio, dichos errores no parecen mantener correlación alguna con los errores observados en las componentes de los cuaterniones. Es decir que esto confirma que los errores surgen al calcular los ángulos de Euler, no al propagar los cuaterniones. De todas formas, nuevamente el método de los cuaterniones parece ser superior frente al de las matrices.

Por último podemos observar si en el método de propagación de matrices, los vectores columna se mantienen ortogonales entre si y con norma unitaria.

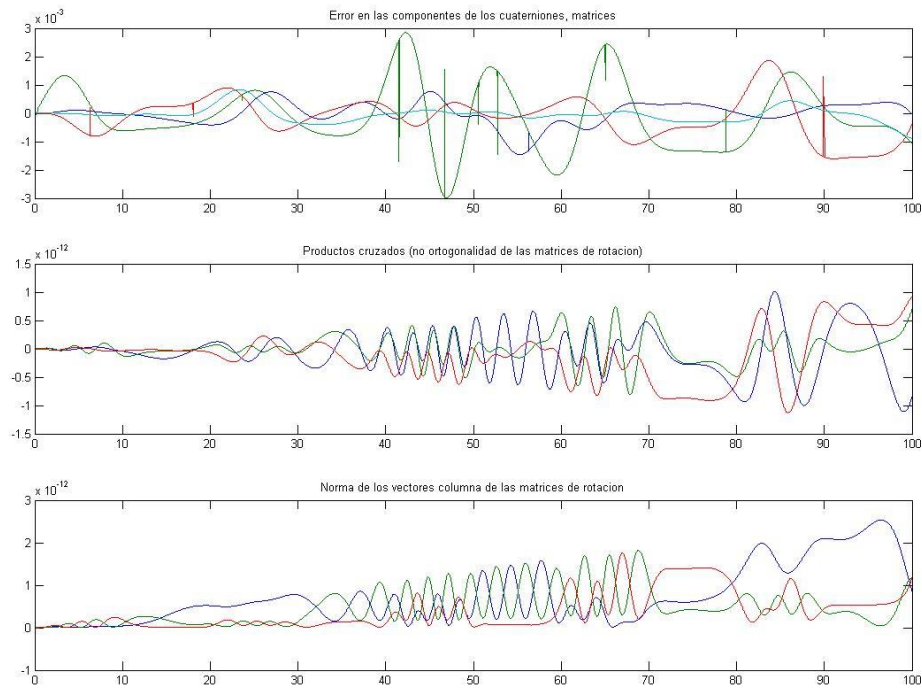


Figura 9 – Ortogonalidad y Normalidad

El primero subgráfico es del error de las componentes de los cuaterniones equivalentes del método de matrices. El segundo corresponde a los productos cruzados de los vectores columna de las matrices propagadas instante a instante. El tercero corresponde a la resta de uno menos la norma de cada vector columna en cada instante de tiempo. Puede observarse la existencia de una tendencia creciente a producirse la no ortogonalidad y norma diferente a la unidad. Esto sin dudas no es algo positivo, ya que si no se pueden asegurar ambas condiciones, las matrices obtenidas dejarían de ser matrices de rotación, y dejarían de ser válidas ciertas propiedades. Esto acarrearía un mayor error en la propagación de las estimaciones de orientación.

Observando estos gráficos a simple vista no se puede afirmar que los únicos efectos que contribuyen al aumento de error en la propagación de este método sean la no ortogonalidad y norma no unitaria de los vectores columna de las matrices, pero con el correr del tiempo, quizás en una simulación más larga, podrían convertirse en factores determinantes.

Conclusiones

En el presente trabajo se desarrolló un método para establecer información de orientación en un vehículo, basándose exclusivamente en mediciones provistas por tres giróscopos ortogonales entre sí.

Se programó un script capaz de sintetizar datos para la posterior prueba de diversos algoritmos de propagación de la orientación.

En un segundo script se implementó el algoritmo en dos versiones: propagación de cuaterniones, y propagación de matrices de rotación. En todos los casos, la versión de los cuaterniones demostró ser superior a la versión de matrices de rotación en cuanto a exactitud en las estimaciones.

Por otro lado quedó en evidencia la desventaja de la utilización de matrices de rotación, y su posible influencia en errores de estimación crecientes con el tiempo, producto de la pérdida de ortonormalidad en las matrices debido a errores numéricos.

Apéndice A: Álgebra de Cuaterniones

Los cuaterniones son conocidos también como la extensión de los complejos a R^3 . Tienen la siguiente forma:

$$q = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_0 \end{bmatrix} = \begin{bmatrix} \bar{\varepsilon} \\ \eta \end{bmatrix}; \quad \bar{\varepsilon} = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix}; \quad \eta = q_0$$

donde $q_1, q_2, q_3, q_0 \in R$

Se definen algunas operaciones básicas que se describen a continuación.

Operaciones

Suma

La suma entre dos cuaterniones puede definirse de la siguiente manera:

$$q_1 + q_2 = \begin{bmatrix} \bar{\varepsilon}_1 + \bar{\varepsilon}_2 \\ \eta_1 + \eta_2 \end{bmatrix}$$

Multipliación

La multiplicación entre dos cuaterniones puede definirse de la siguiente manera:

$$q_1 \cdot q_2 = \begin{bmatrix} \eta_1 \cdot I_3 + S(\bar{\varepsilon}_1) & \bar{\varepsilon}_1 \\ -\bar{\varepsilon}_1^T & \eta_1 \end{bmatrix} \cdot \begin{bmatrix} \bar{\varepsilon}_2 \\ \eta_2 \end{bmatrix} = \begin{bmatrix} \eta_1 \cdot \bar{\varepsilon}_2 + \eta_2 \cdot \bar{\varepsilon}_1 + \bar{\varepsilon}_1 \times \bar{\varepsilon}_2 \\ -\bar{\varepsilon}_1^T \cdot \bar{\varepsilon}_2 + \eta_1 \cdot \eta_2 \end{bmatrix}$$

$(H, +, \cdot)$ Anillo no conmutativo.

Opuesto

El opuesto de un cuaternión puede definirse de la siguiente manera:

$$\bar{q}_1 = \begin{bmatrix} -\bar{\varepsilon}_1 \\ -\eta_1 \end{bmatrix}; \quad \forall q \exists \bar{q} / q + \bar{q} = 0$$

Conjugado

El conjugado de un cuaternión puede definirse de la siguiente manera:

$$q^* = \begin{bmatrix} -\bar{\varepsilon}_1 \\ \eta_1 \end{bmatrix}; \quad \forall q \exists q^{-1} = \frac{q^*}{\|q\|^2} \quad \Bigg/ \quad q \cdot q^{-1} = q^{-1} \cdot q = q_I = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

donde $\|q\| = \sqrt{q_1^2 + q_2^2 + q_3^2 + q_0^2}$

Relación con eje y ángulo de Euler

Dado un eje y ángulo de Euler: $\bar{\phi} = \phi \cdot n$ se define:

$$q = \begin{bmatrix} n \cdot \sin\left(\frac{\phi}{2}\right) \\ \cos\left(\frac{\phi}{2}\right) \end{bmatrix} \rightarrow \|q\| = 1$$

El cuaternión definido representa una rotación. Para rotar un vector de R^3 cualquiera según el eje y ángulo de Euler debe realizarse la siguiente operación:

$$\bar{\phi}_{ba} \rightarrow \text{ángulo rotado de "a" a "b"}$$

$$\begin{bmatrix} v^a \\ 0 \end{bmatrix} = q_b^a \cdot \begin{bmatrix} v^b \\ 0 \end{bmatrix} \cdot (q_b^a)^*$$

Rotaciones elementales de Euler

Las rotaciones elementales de Euler son rotaciones alrededor de los ejes de la terna. Para definir un cuaternión de rotación alrededor de un eje (por ejemplo el eje x), debe definirse el siguiente cuaternión de rotación y realizarse la operación descrita anteriormente para rotar vectores:

rotación : $(\alpha @ x)$

$$\Rightarrow q = \begin{bmatrix} -\sin\left(\frac{\alpha}{2}\right) \\ 0 \\ 0 \\ \cos\left(\frac{\alpha}{2}\right) \end{bmatrix}$$

Bibliografía / Referencias

1. "Fundamentos de la Navegación Integrada"; Martín España.
- Seminario de Navegación Integrada, Facultad de Ingeniería de la Universidad de Buenos Aires; Martín España y Juan Carrizo.